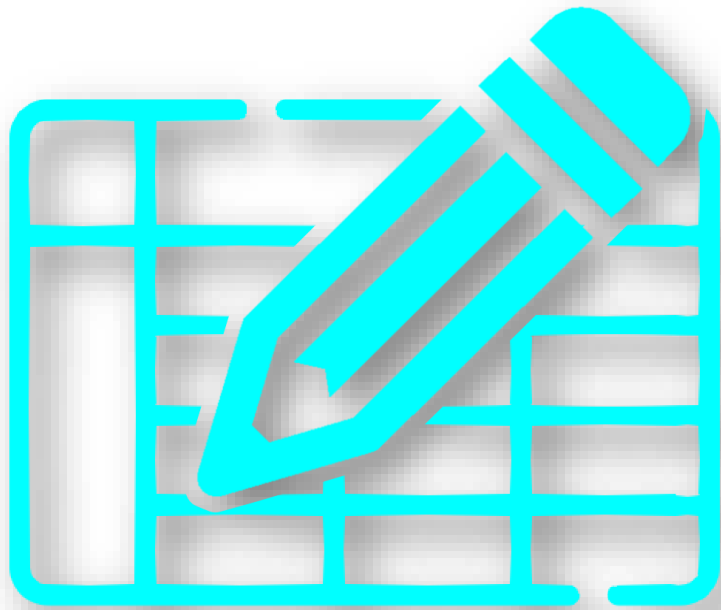




TSV Tools

Documentation

Creator: Lucas Gomes Cecchini

Online Name: AGAMENOM

Overview

TSV Tools is a tool for creating and modifying **.tsv** files for **Unity**.

With it, you can create a text-based data table in the **editor**, and it also allows you to read and write or edit the file in both **Editor** and **Build** mode.

Script Explanations

Tab Table Utility

This script is a **general-purpose TSV (tab-separated values) table manager**.

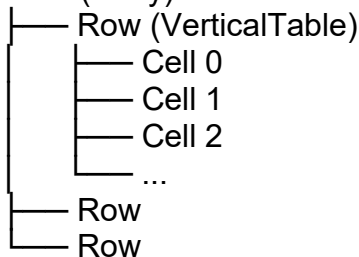
Think of it as a **mini spreadsheet system inside Unity**, but simplified and focused on:

- Loading/saving .tsv files
- Editing rows and columns
- Accessing individual cells
- Handling special characters safely

1. Core Concept

The table structure is:

Table (array)



- The **table** is a list of rows
- Each **row** contains multiple cells (strings)

👉 “Vertical” = rows

👉 “Horizontal” = columns

2. Enum (Direction Control)

LineDirection

This defines how operations behave:

- **Vertical**
 - Works with rows
 - Add/remove entire rows
- **Horizontal**
 - Works with columns
 - Add/remove cells across all rows

👉 This is used when modifying the table.

3. Text Conversion System

ConvertText(text)

This is a **two-way converter** for special characters.

Why it exists:

TSV files use:

- tab → separates columns
- newline → separates rows

If your text contains these, it breaks the table.

What it does:

Case 1: Text contains tokens

%tab% → becomes real tab

%newline% → becomes real newline

Case 2: Text contains real characters

tab → becomes %tab%

newline → becomes %newline%

- 👉 This ensures:
- File stays valid
 - Text formatting is preserved

📁 4. File Operations

📄 LoadTableFile(filePath, table)

What it does:

Loads a .tsv file into memory.

Step-by-step:

1. Checks if file exists
2. Opens file with encoding detection
 - Supports UTF-8 with or without BOM
3. Reads line by line
4. Splits each line using tab
5. Converts each cell using ConvertText
6. Creates rows and stores them
7. Converts list into final table array

Result:

You get a structured table in memory.

💾 SaveTableFile(filePath, table)

What it does:

Saves the table back into a .tsv file.

Step-by-step:

1. Validates table is not empty
2. Ensures folder exists (creates if needed)
3. Finds the largest row (max columns)
4. Normalizes all rows:
 - Fills missing cells with empty text
5. Converts each row into a tab-separated line
6. Converts text using ConvertText
7. Saves file using UTF-8 (no BOM)

Important detail:

All rows will always have the same number of columns after saving.

📊 5. Table Access

🔍 GetText(table, row, column)

What it does:

Returns the content of a specific cell.

Safety checks:

- Table exists
- Row index is valid
- Column index is valid

Extra feature:

- Detects **who called the function**
- Adds that info to error logs

👉 Very useful for debugging.

SetText(table, row, column, text)

What it does:

Changes the content of a specific cell.

Safety checks:

- Same as GetText

Behavior:

- Directly replaces the value in that cell

6. Table Modification

AddLine(table, index, direction)

Adds either a row or a column.

Vertical (Row):

- Creates a new row
- Same number of columns as existing rows
- All cells start empty
- Inserts at specified position

Horizontal (Column):

- Goes through every row
- Inserts a new empty cell
- At the specified column index

Result:

- Table grows either downward or sideways

RemoveLine(table, index, direction)

Removes either a row or a column.

Vertical (Row):

- Removes entire row at index

Horizontal (Column):

- Removes that column from all rows

Safety checks:

- Index must be valid
- Rows must be consistent

Result:

- Table shrinks accordingly

7. Data Structure

VerticalTable

This represents a **single row**.

Contains:

- **horizontalTable** → array of strings (cells)

Example row:

["Hello", "World", "123"]

8. How You Actually Use This

✓ Load a table

- Call load function
- Get structured data

✓ Read data

- Use `GetText(row, column)`

✓ Modify data

- Use `SetText`
- Add/remove rows or columns

✓ Save back

- Call save function

✓ Handle special text

- Automatically handled via `ConvertText`

⚠ Important Behaviors

- Rows can temporarily have different sizes
- But saving will **force consistency**
- Text is always sanitized for TSV safety
- Errors are logged instead of crashing

🧠 Final Summary

This script is essentially:

A lightweight spreadsheet engine for TSV files inside Unity

It gives you:

- File I/O (load/save)
- Safe text handling
- Full table editing (rows + columns)
- Cell-level access
- Debug-friendly validation

TSV Editor Window

This script is a **visual TSV table editor inside Unity**.

If the previous utility script was the “engine,” this one is the **user interface**—a full spreadsheet-like editor with:

- Scrollable table view
- Editable cells
- Resizable rows and columns
- Add/remove rows and columns
- File loading and saving

🧠 1. Core Idea

This window lets you **open, edit, and save TSV files visually**, almost like a lightweight Excel inside Unity.

It uses the table system from your utility script and adds:

- Interaction (mouse, clicks, drag)
- Rendering (drawing cells, headers, grid)
- Editing (typing inside cells)

2. Main Data (Public Variables)

These are the **core data structures and settings**:



Table Data

- **tableData**
The actual table.
Each element = one row
Each row contains multiple cells



File Info

- **filePath**
Stores the path of the currently loaded file



Editing Controls

- **targetIndex**
The selected row or column index for operations (like removal)
- **removalDirection**
Defines whether you're working with:
 - rows (vertical)
 - columns (horizontal)



Layout Sizes

- **rowSizes**
Stores the height of each row
- **columnSizes**
Stores the width of each column



These allow resizing like a spreadsheet.



3. Interaction State (Internal Behavior)

These variables track what the user is doing:



Resizing

- **isResizingRow / isResizingColumn**
True when user is dragging to resize
- **initialResizePosition**
Mouse position when resizing started
- **originalDimension**
Original size before resizing
- **resizeTargetIndex**
Which row/column is being resized



Navigation

- **mainScrollPosition** → scroll of entire window
- **tableScrollPosition** → scroll inside table



State Tracking

- **isDirty**
True when changes were made but not saved
- **previousScrollPosition**
Used to detect scrolling changes



Interaction Timing

- **lastInteractionTime**
Used to detect double-clicks



Validation

- **isValidIndex**
Ensures selected row/column is valid



4. Visual Settings

These define how the editor looks:



Limits

- **minColumnWidth** → minimum column size
- **minRowHeight** → minimum row size



Double Click

- **doubleClickThreshold**
Time window to detect double-click



Colors

- Default grid color
- Highlight color (when resizing)
- Background colors
- Selected row/column color



Styles

- **cellTextStyle** → how text looks inside cells
- **headerLabelStyle** → row/column labels

5. Window Creation

OpenTableEditorWindow

What it does:

- Opens the editor from:
- Tools → TSV Editor Window
- Searches for an icon
- Sets window title and icon



6. Lifecycle (Unity Events)

OnEnable

- Activates undo/redo tracking

OnDestroy

- Saves a temporary backup file
- Removes event listeners

OnUndoRedo

- Refreshes the window after undo/redo



7. Main Interface Loop

OnGUI

This runs constantly and draws everything.

It does:

1. Initializes styles
2. Creates scrollable window
3. Shows:
 - Title
 - Open file button

- Current file path
- 4. If table exists:
 - Show Save button
 - Show remove controls
 - Draw table
- 5. If no table:
 - Show “Create Table” button

8. UI Components

InitializeStyles

Creates:

- Cell background textures
- Text styles
- Header styles

RenderOpenFileButton

- Opens file dialog
- Loads TSV file
- Converts into table

RenderSaveButton

- Shows save button
- Highlights if unsaved changes exist
- Opens save dialog
- Saves file

RenderRemoveRowColumnControls

Lets user:

- Choose index
- Choose direction (row/column)
- Remove selected line

Also:

- Validates index
- Shows warnings

RemoveSelectedLine

- Removes row or column
- Updates size arrays
- Marks table as modified

RenderCreateTableButton

Creates a default table:

- 2 rows
- 2 columns
- Default sizes

9. File Operations (Bridge to Utility Script)

LoadTableFromFile

- Loads TSV file
- Initializes row and column sizes

SaveTableToFile

- Saves table using utility system
- Clears “dirty” state
- Shows success message

10. Table Rendering (THE CORE)

RenderTable

This is the **heart of the system**.

Step 1: Calculate Size

- Total width = sum of column widths
- Total height = sum of row heights

Step 2: Setup Scroll View

- Creates visible area
- Enables scrolling
- Tracks visible region

Step 3: Draw Column Headers

For each column:

- Draw header box
- Show column index
- Highlight if selected
- Add resize handle



Interaction:

- Click → start resizing
- Double-click → add new column

Step 4: Draw Rows

For each row:

- Draw row label
- Highlight if selected

Step 5: Draw Cells

For each cell:

- Draw background
- Draw editable text field



Editing:

- When text changes:
 - Update data
 - Mark as dirty

Step 6: Resize Handles

Each cell edge has:

- Column resize handle
- Row resize handle



Behavior:

Drag:

- Resizes row or column

Double-click:

- Adds new row or column



11. Resize System

HandleResizeInteraction

Handles dragging behavior:

While dragging:

- Updates size based on mouse movement
- Ensures minimum size

On release:

- Stops resizing

+ 12. Dynamic Table Editing

Add row:

- Double-click horizontal line

Add column:

- Double-click vertical line

Remove:

- Use control panel

13. How You Use This Tool

✓ Open TSV

- Click “Open File”
- Select .tsv

✓ Edit Cells

- Click cell
- Type text

✓ Resize

- Drag borders

✓ Add Rows/Columns

- Double-click edges

✓ Remove Rows/Columns

- Enter index
- Choose direction
- Click remove

✓ Save

- Click “Save Table”

Important Behavior

- Changes are tracked (dirty state)
- Undo/Redo works
- Table is fully dynamic
- UI only renders visible cells (optimized)

Final Summary

This script is:

A full visual TSV editor with spreadsheet-like interaction inside Unity

It combines:

- Data system (from utility script)
- UI rendering
- Mouse interaction
- File management

Test Table

This script is a **simple testing tool for your TSV system inside Unity's Inspector**.

Unlike the full editor window (which behaves like a spreadsheet), this one is more like a **control panel with buttons** to quickly test:

- Loading a TSV file
- Saving it
- Reading a specific cell
- Writing to a cell
- Adding/removing rows or columns

Think of it as a **debug/utility component** you attach to an object.

1. Core Idea

You add this component to a GameObject, and it gives you **buttons in the Inspector** to interact with a TSV table without needing a custom window.

It uses the functions from your table utility system and exposes them in a very direct way.

2. Main Data (Serialized Fields)

These are the values you see and edit in the Inspector.

Table Data

- **table**
The actual table loaded in memory.
Each element = one row
Each row = multiple cells

File Path

- **filePath**
Stores the location of the TSV file you load or save

Cell Editing

- **text**
Temporary text used when reading or writing a cell
- **columnVertical**
Row index (which row)
- **columnHorizontal**
Column index (which column)

 Together, these define a specific cell.

Add / Remove

- **direction**
Defines operation type:
 - Vertical → rows
 - Horizontal → columns
- **addColumn**
The index where a row/column will be added or removed

3. Properties (Access Wrappers)

These simply expose the internal values in a controlled way:

- Table
- FilePath
- Text
- ColumnVertical

- ColumnHorizontal
- Direction
- AddColumn

👉 They allow external systems to read/write safely if needed.

⚙️ 4. Core Methods (What Each Button Does)

📁 GetFile

What it does:

- Opens a file picker
- Lets you choose a .tsv file
- Loads it into memory

Flow:

1. Opens file dialog
2. If file selected:
 - Loads table
 - Stores path
 - Logs success
3. If canceled:
 - Shows warning

👉 This is the **entry point** for using the table.

✅ Checking

What it does:

- Verifies if a table exists before operations

Behavior:

- If table is null or empty:
 - Logs error
 - Stops operation

👉 Prevents crashes and invalid actions.

💾 SaveFile

What it does:

- Saves the current table to a file

Flow:

1. Checks if table exists
2. Opens save dialog
3. If confirmed:
 - Writes file
 - Logs success
4. If canceled:
 - Logs warning

👉 This is how you persist changes.

🔍 GetText

What it does:

- Reads a specific cell

Flow:

1. Checks table
2. Reads cell using row + column
3. Stores result in **text field**
4. Logs value

👉 Useful for debugging or inspecting data.

SetText

What it does:

- Writes text into a specific cell

Flow:

1. Checks table
2. Takes value from **text field**
3. Writes into selected cell
4. Logs result

👉 This is your manual editing tool.

AddColumnInTable

What it does:

- Adds a row or column

Behavior:

- Uses:
 - **direction** (row or column)
 - **addColumn** (position)

👉 Example:

- Add row at index 2
- Add column at index 1

RemoveColumnInTable

What it does:

- Removes a row or column

Behavior:

- Same logic as add, but removes instead

👉 Example:

- Remove row 0
- Remove column 3

5. Custom Inspector (Buttons UI)

This is what makes the script practical.

TestTableInspector

This replaces the default Inspector UI with buttons.

Layout:

Row 1: File operations

- Get File
- Save File

Row 2: Cell operations

- Get Text
- Set Text

Row 3: Table structure

- Add Line
- Remove Line

Then:

- Shows default fields below (table, indices, text, etc.)



So you:

1. Configure values
2. Click buttons
3. See results instantly



6. How You Actually Use This



Step-by-step workflow

1. Add component

Attach it to any GameObject.

2. Load a file

Click:

Get File

3. Read a cell

- Set row + column
- Click:

Get Text

4. Modify a cell

- Change "text" field
- Click:

Set Text

5. Add/remove structure

- Set index
- Choose direction
- Click:

Add Line / Remove Line

6. Save changes

Click:

Save File



Important Behavior

- No visual table (unlike the editor window)
- Fully manual via fields + buttons
- Requires valid indices
- Uses utility system internally
- Safe (checks before operations)



Final Summary

This script is:

A debug/testing interface for TSV tables directly in the Inspector

It provides:

- Quick access to table operations
- Manual control over data
- Easy testing without UI complexity